Assignment 03

Edge Computing & Intelligent Systems

23i-3022 | Section A

Model Quantization & Neural Network Pruning

ResNet-18 on CIFAR-10

1. Introduction

This report documents the implementation of two model optimization techniques for deploying a ResNet-18 image classification model on edge devices:

1. Quantization (Question 1): Converting FP32 weights and activations to INT8 via Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT).

2. Pruning (Question 2): Reducing model complexity via unstructured magnitude-based pruning and structured channel pruning.

All experiments use CIFAR-10 (10 classes, 32x32 RGB images, 50K training / 10K test). The complete pipeline is implemented in a single Kaggle notebook running on GPU T4 x2.

2. Model Architecture

We use ResNet-18 modified for CIFAR-10's 32x32 input resolution:

- Initial convolution: 3x3 kernel, stride=1, padding=1 (instead of 7x7 stride=2)
- No max pooling after the initial convolution
- Four residual layer groups: [64, 128, 256, 512] channels
- Total parameters: 11,173,962 (FP32 baseline)

For quantization, the model includes QuantStub/DeQuantStub at input/output and FloatFunctional for residual additions, enabling proper quantized inference. For structured pruning, a configurable channels parameter allows reconstructing a genuinely smaller model with fewer channels per layer.

3. Question 1: Model Quantization

3.1 Baseline Training

The baseline FP32 model was trained for 100 epochs using:

- Optimizer: SGD (momentum=0.9, weight decay=5e-4)
- Learning rate: 0.1 with cosine annealing
- Data augmentation: Random crop (padding=4), horizontal flip
- Achieved test accuracy: 94.83%

3.2 Post-Training Quantization (PTQ)

Pipeline:
1. Load pretrained FP32 model with QuantStub/DeQuantStub
2. Fuse Conv-BN-ReLU layers into single modules
3. Configure x86 quantization backend
4. Insert observers via torch.ao.quantization.prepare()
5. Calibrate with 1000 training samples (no augmentation)
6. Convert to INT8 via torch.ao.quantization.convert()

Layer fusion combines Conv2d + BatchNorm2d + ReLU into ConvBnReLU2d. Calibration collects activation statistics (min/max ranges) needed for mapping FP32 values to INT8.

Result: 94.77% accuracy (only 0.06% drop from FP32), model size reduced from 42.7 MB to 10.78 MB (4x compression), inference latency reduced from 14.33 ms to 7.0 ms (2x speedup).

3.3 Quantization-Aware Training (QAT)

Pipeline:
1. Load pretrained FP32 model
2. Fuse layers in training mode (fuse_modules_qat)
3. Insert fake quantization modules via prepare_qat()
4. Fine-tune for 10 epochs with learning rate 1e-3
5. Freeze observers after epoch 3 to stabilize quantization ranges
6. Convert to INT8 via convert()

Fake quantization simulates INT8 rounding during the forward pass while maintaining full-precision gradients in the backward pass. This allows the model to learn to compensate for quantization error.

Result: 94.80% accuracy (only 0.03% drop from FP32), same 4x compression and 2x speedup as PTQ but with better accuracy.

4. Question 2: Neural Network Pruning

4.1 Unstructured Pruning

Approach: Global L1 unstructured pruning with iterative rounds.

Pipeline:
1. Load pretrained FP32 model
2. For each pruning iteration (3 rounds):
   a. Apply global L1 pruning across all Conv2d and Linear layers
   b. Target sparsity follows cubic schedule: s = 1 - (1 - s_target)^(i/N)
   c. Fine-tune for 10 epochs after each round
3. Make pruning permanent (remove masks and hooks)

Global pruning considers all layers together, pruning the smallest magnitude weights regardless of which layer they belong to. The iterative approach with fine-tuning between rounds preserves more accuracy than one-shot pruning.

Result: 94.53% accuracy with 90.9% sparsity (exceeded 70% target). Model file size remains 42.7 MB in dense format. Sparse formats or sparse hardware acceleration would be needed for actual storage/speed benefits.

4.2 Structured Channel Pruning

Approach: L1-norm channel importance with full model reconstruction.

Pipeline:
1. Compute per-channel importance using L1 norm of conv2 filters, aggregated across all blocks in each layer group
2. Select top-k channels to keep (k = original * (1 - prune_ratio))
3. Reconstruct a new, smaller ResNet-18 with pruned channel dimensions
4. Copy weights for kept channels, handling cross-layer dependencies
5. Fine-tune for 20 epochs

Channel dependency chain:
conv1 [3->c0] -> layer1 [c0->c0] -> layer2 [c0->c1] -> layer3 [c1->c2] -> layer4 [c2->c3] -> FC [c3->10]

New channel dimensions: [44, 89, 179, 358] (from [64, 128, 256, 512]).

Result: 94.36% accuracy with 5,459,866 parameters (51% reduction from 11.2M). Model size: 20.89 MB (51% smaller). Inference latency: 11.11 ms (22% faster). Unlike unstructured pruning, the reconstructed model is genuinely smaller and runs faster on any hardware without sparse computation support.

5. Experimental Results

5.1 Summary Table

Model	Acc (%)	Size (MB)	Latency (ms)	Params
FP32 Baseline	94.83	42.70	14.33	11,173,962
PTQ INT8	94.77	10.78	7.00	-
QAT INT8	94.80	10.78	7.08	-
Unstructured	94.53	42.70	14.11	11,173,962
Structured	94.36	20.89	11.11	5,459,866

5.2 Quantization Results

Both PTQ and QAT achieve near-lossless INT8 quantization:

- PTQ accuracy drop: 0.06% (94.83 -> 94.77)
- QAT accuracy drop: 0.03% (94.83 -> 94.80)
- Model size: 4x reduction (42.7 MB -> 10.78 MB)
- Latency: 2x speedup (14.33 ms -> 7.0 ms)

QAT recovers 0.03% more accuracy than PTQ by learning to compensate for quantization noise during training.

5.3 Pruning Results

- Unstructured: 90.9% sparsity with only 0.30% accuracy drop. Same file size in dense format but significant storage savings with sparse representations.

- Structured: 51% parameter reduction with only 0.47% accuracy drop. Genuinely smaller model (20.89 MB vs 42.7 MB) with real speedup (11.11 ms vs 14.33 ms) on standard hardware.

5.4 Latency Percentiles (CPU)

Model	Mean (ms)	P50 (ms)	P95 (ms)	P99 (ms)
FP32	14.33	14.15	16.28	17.31
PTQ INT8	7.00	6.96	7.50	8.26
QAT INT8	7.08	7.03	7.56	8.41
Unstructured	14.11	14.09	14.94	15.38
Structured	11.11	10.94	12.04	15.18

6. ONNX Deployment

FP32, unstructured-pruned, and structured-pruned models were exported to ONNX format (opset 13) with dynamic batch size support and constant folding optimization. All models were validated with onnx.checker and verified using ONNX Runtime CPUExecutionProvider.

ONNX Runtime inference accuracy matches PyTorch exactly:
- FP32: 94.83%
- Unstructured: 94.53%
- Structured: 94.36%

7. Conclusions

1. QAT vs PTQ: QAT achieves slightly higher accuracy (94.80% vs 94.77%) because the model adapts to quantization noise during training. Both provide 4x compression and 2x speedup.

2. Structured vs Unstructured Pruning: Structured pruning provides real speedup (22% faster) by reducing tensor dimensions. Unstructured pruning achieves higher sparsity (90.9%) but requires sparse hardware for actual acceleration.

3. Iterative Pruning: Gradual pruning with fine-tuning between rounds preserves significantly more accuracy than one-shot pruning.

4. All optimization techniques maintain accuracy above 94%, demonstrating that significant model compression is possible with minimal quality loss for edge deployment scenarios.